

Every Line Is a Feature Request

What Generic Guidance Buys in an Agent Prompt, and What It Costs

Jashwanth Lakshmi Sai Yenugu
Independent Researcher
yenugujashwanth@gmail.com
<https://orcid.org/0009-0008-1831-2615>

September 8, 2026

Abstract

System prompts and agent skills accumulate general guidance — write semantic HTML, respect reduced-motion, use custom properties, handle empty states — alongside the project-specific facts a model cannot infer. The guidance is correct, and a frontier model knows all of it. We ask what it costs to include, and what it buys.

Across 172 runs on two OpenAI GPT-5.6 models under pinned reasoning effort, we append 0 to 811 words of such guidance to otherwise fixed build tasks and hold everything else constant. Three results follow. **It costs.** Output tokens rise 42% and latency 38% at 811 words, in every one of twelve paired cells ($p = 0.00024$), and on an open-ended task 811 words doubled latency. **It buys exactly what it asks for.** The model does not apply the guidance unprompted: a reduced-motion query appears in 0 of 12 unpadded files and 12 of 12 padded ones, ARIA attributes double, `rem` units rise eightfold, and on the open-ended task the model writes six times more JavaScript. Generic guidance is not a reminder the model already honours; it is a set of feature requests the model fills and bills for. **Correctness does not move.** Behavioural correctness and rule-following hold at ceiling across three increasingly demanding instruments, and a positive control confirms the pipeline registers a 26-point drop when a single needed fact is removed. Cost is not a correctness trade in either direction.

Separating the two things people put in skills, a line that *asks for work* is billed whether it is phrased as guidance or as a rule; a line that *restricts* is free. We contribute a three-control validation protocol for behavioural graders — a correct reference must score 100%, an empty page 0%, and a single-rule mutation must fail exactly its own checks — with six defects it catches, each of which had produced a clean and wrong number. The practical rule is short: treat every generic line in a prompt as a purchase order, keep the ones whose output you want, and cut the rest.

1 The question

A production skill for lesson generation runs to 843 lines. A contracts-only variant that keeps every project-specific requirement — the house component names, the required output structure, the conventions no model could guess — fits in 154. The remainder is general guidance of the kind any current frontier model can recite: semantic elements, ordered headings, labelled controls, named constants, graceful failure.

Two beliefs about that remainder are common and opposed. One holds that it is harmless insurance. The other holds that it is bloat that distracts the model and degrades the work. Neither is usually measured. We measure three things about it:

- **Cost.** What does it add to tokens and latency?
- **Return.** Does the model's output change, and how?
- **Correctness.** Does anything the task requires get worse?

The answers turn out to be: a lot, yes in a specific way, and no.

2 Design

Guidance ladder. Four cumulative levels of general web-development guidance, fixed in code before any run (`experiments/dose/design.py`). Every statement is correct, generic, and free of task-specific facts; none of it is wrong, and none of it names anything a task requires.

level	words	adds
L0	0	nothing
L1	68	semantic elements, heading order, form labels, contrast, keyboard reachability, focus indicators
L2	246	CSS conventions (flat specificity, custom properties, <code>rem</code> , <code>flex/grid</code> , responsive) and JavaScript conventions (<code>const/let</code> , <code>addEventListener</code> , small functions, named constants)
L3	811	performance, maintainability and comments, accessibility beyond the basics (<code>aria-live</code> , tab order, reduced-motion), robustness and empty states, state management, error handling, user feedback

Tasks. Two self-contained HTML builds for the ladder — a five-question quiz with gated submission and full retry, and a sortable, filterable employee table with pagination and CSV export — each graded behaviourally by loading the page in `jsdom` and driving it like a user. Later experiments add a five-rule compliance task, three deliberately hard calibration tasks, a pricing-engine positive control, and an open-ended dashboard brief.

Models and harness. `gpt-5.6-luna` and `gpt-5.6-terra` (OpenAI; all runs 7 September 2026), reasoning effort pinned to `medium` through the Responses API with text verbosity at its default, run through LangChain Deep Agents with a filesystem backend. Effort is pinned because provider-default effort differs between models and materially changes what a model produces — an observation from our earlier audit of prompting guidance across model generations — so leaving it floating would confound any comparison between prompts. Deep Agents prepends its own base prompt and tool schemas to every call, so the "zero guidance" arm is zero *added* guidance, not an empty system prompt. Reported output tokens are the API's `output_tokens`, which include reasoning; file size is reported alongside as the visible measure. Spend is estimated from list prices (\$1.25 per million input, \$10 per million output), not billed.

Scale. 172 runs, \$14.78: a 16-run direct-API pilot, a 20-run Deep Agents replication, a 48-run guidance ladder, a 24-run compliance test, a 12-run hard-task calibration, a 12-run positive control, a 36-run diversity experiment, and a 4-run pilot on the motivating skill. Every design was fixed in code before its runs; we do not claim external registration. Every numeric claim in this paper is re-derived from the raw logs by `experiments/verify_paper.py`, which exits non-zero on any mismatch.

3 What generic guidance costs

Pooled across both tasks and both models, $n = 12$ per level:

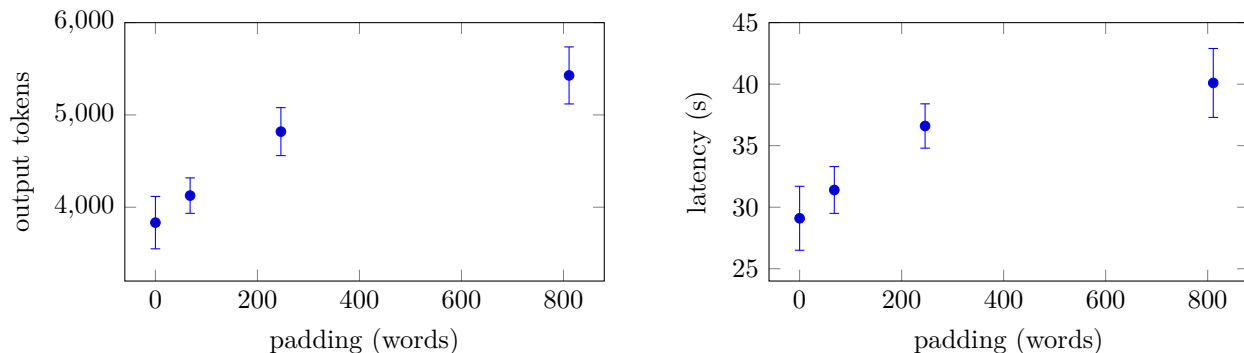


Figure 1: Cost rises monotonically with padding that adds no information. Output tokens (left) and end-to-end latency (right) against the quantity of general-knowledge padding, pooled over two tasks and two models, $n = 12$ per dose. Bars are 95% bootstrap intervals. The padding is *input*, yet it is the output that grows by 42%. Comparing within each task/model/trial cell, the increase holds in all twelve pairs (sign test $p = 0.00024$).

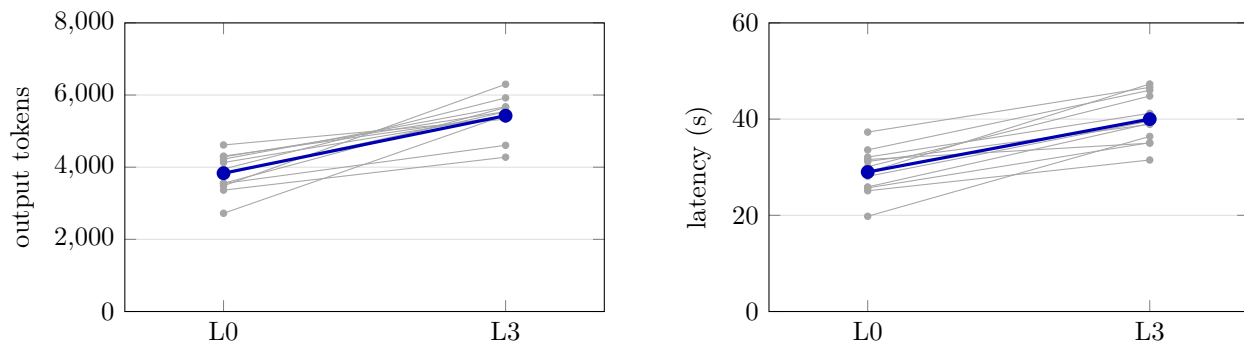


Figure 2: Every pair rises. Each grey line is one task, model and trial at zero guidance (L0) and 811 words (L3); the blue line is the mean. Output tokens rise in 12 of 12 pairs and latency in 12 of 12, one-sided sign test $p = 0.00024$ for both.

level	words	output tokens	latency	file size
L0	0	3,833	29.1s	12.1k
L1	68	4,126 (+8%)	31.4s (+8%)	
L2	246	4,819 (+26%)	36.6s (+26%)	
L3	811	5,428 (+42%)	40.1s (+38%)	18.2k (+50%)

Figure 1 plots both. The relationship is monotonic in both columns, and the endpoints are well separated: 95% bootstrap intervals put L0 output at 3,833 [3,549, 4,105] tokens against L3's 5,428 [5,119, 5,701], and latency at 29.1 [26.6, 31.6] seconds against 40.1 [37.3, 42.9].

Every pair rises. Comparing L0 to L3 within each task, model and trial rather than across pooled means, the increase holds in all twelve pairs (Figure 2): output +1,595 [+1,241, +1,989] tokens and latency +11.0 [+8.7, +13.3] seconds, one-sided sign test $p = 0.00024$ for both. Output tokens explain most of the latency variance, and file size grows 50%, so the growth is visible output rather than hidden reasoning.

The bill is paid at the output. The guidance is 811 words of *input*, yet the measurable cost lands on the output side: the model ships 42% more tokens and 50% more file. At the list prices used here output is priced eight times input, so the second payment is the one that matters — and, unlike input, it is not reduced by prompt caching.

Pilots at other doses. A first pilot (429 words of guidance, $n = 4$ per cell direct against the API and $n = 5$ through Deep Agents) found latency +40% and +51% direct, +45% and +85% through the agent, for luna and terra respectively. Turn counts did not change between arms

4 What it buys

If the model already applied the guidance unprompted, its output would not change when the guidance was added. It does — and in exactly the places the guidance names. A content audit of the same files (`experiments/audit/content.py`) counts, per file, the constructs each directive asks for.

directive in the guidance	L0, per file	L3, per file	open-ended task, minimal → padded
respect prefers-reduced-motion	0 of 12 files	12 of 12 files	0 → 12 of 12
ARIA attributes	7.3	14.1	2.5 → 25.1
rem units	4.8	38.5	0.0 → 81.2
CSS custom properties	7.9	18.8	12.3 → 25.4
semantic elements	3.7	7.3	8.8 → 14.7
const/let declarations	22.5	36.5	7.3 → 39.3
addEventListener	4.5	4.6	0.6 → 4.2
inline onclick , var	0	0	0 → 0
try/catch , code comments	0	0	0 → 0

Three patterns appear. Some guidance was **already followed** at zero padding: no inline handlers, no **var**, listeners attached properly. Some was **adopted only when asked**: reduced-motion support went from absent to universal, ARIA doubled, **rem** units rose eightfold, custom properties more than doubled. Some was **ignored either way**: the guidance asks for comments and graceful error handling, and no file at any level contains a comment or a **catch**. On the open-ended dashboard, where the brief left the most room, the model wrote six times more JavaScript under guidance (1,272 → 7,939 characters) while markup stayed flat: it built interaction and accessibility scaffolding it otherwise omitted.

This is the mechanism behind §3. The guidance is not a reminder of behaviour the model already exhibits; it is a checklist the model works through. Each item it adopts is a feature the output did not have before, and each is paid for in tokens. Whether that is a return or a cost depends entirely on whether the team wanted the feature. A team that cuts accessibility guidance on the strength of "the model knows this" will lose the ARIA it was getting.

5 Correctness holds at ceiling

The cost is not a correctness trade. We tested for degradation with three instruments of increasing sensitivity; correctness held at ceiling on every one.

Behavioural correctness. Of the 48 ladder outputs, 47 pass every check; the single defect — a `RadioNodeList` has no `.closest()` — occurred at zero guidance. Both task families sit at ceiling.

Rule-following. Five mechanically checkable house rules were stated once, before the task, with the ladder appended after them. The rules were kept at every level: one file at L1 placed a hex colour outside `:root`, one carried an unprefixed id, and nothing else was ever broken. Three of the five rules were never broken under any condition.

Deliberately hard tasks. Because both instruments were at ceiling, we wrote three tasks to be difficult — a formula engine with cycle detection and undo, a pricing engine with non-stacking coupon precedence, a form with exact cross-field error strings — and calibrated them at zero guidance against an admission band of 55–80% fixed in advance. They scored 95.5%, 100% and

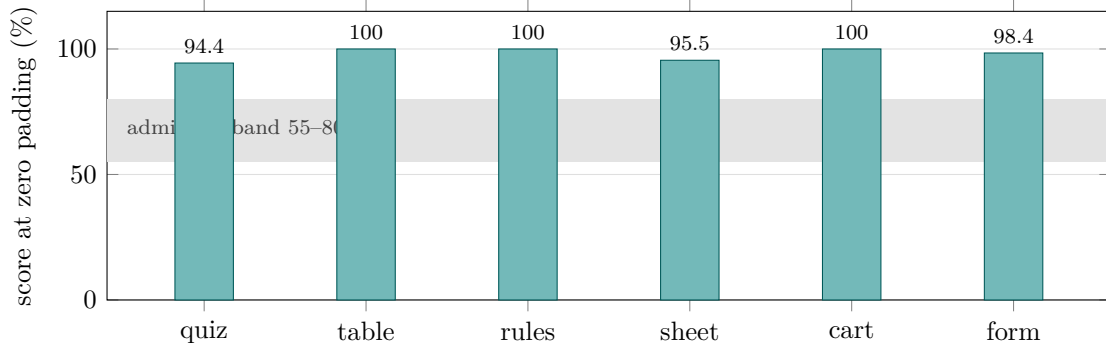


Figure 3: Every instrument sat at ceiling. Zero-padding scores for the six task instruments across three attempts: the two dose-ladder builds, the five-rule compliance test, and the three deliberately hard calibration tasks. The shaded band is the pre-registered 55–80% admission range that a task had to fall into before being carried to the ladder. The lowest score is 94.4%; nothing entered the band, and the band was not moved.

98.4% across 22, 80 and 31 checks. Nothing entered the band, and the band was not moved. Every instrument sat at ceiling (Figure 3).

The pipeline is sensitive. A ceiling result means something only if the instrument can move, so we ran a positive control: the pricing task, the same grader, the same model and effort, with one parameter the model cannot infer removed while the requirement to display the value is kept. Success was defined in advance as any ablated arm below 80%.

arm	removed	score
full (control)	—	100.0%
minus tiers	the 5/10/15% thresholds	91.6%
minus coupon	the non-stacking precedence rules	91.2%
minus tax	the 8.25% rate	73.8%

Where the failures land. In the minus-tax arm only tax and total checks failed — all 14 of each, in three of four runs; Figure 5 — while every subtotal, discount and shipping check still passed. (Figure 4) The grader isolates the removed fact and nothing else. Models sometimes guessed conventional parameters (5/10/15% tiers in two runs of four; 8.25% tax once), so conventional facts are partly recoverable from priors and arbitrary ones are not.

What this licenses. The pipeline registers a large, targeted loss of information as a 26-point drop. It does not establish power against a small, diffuse effect: with at most one failure in twelve runs per level, a per-run failure rate below roughly 25% would not have been detected. The claim is therefore bounded and stated as such — adding up to 811 words of guidance produced no degradation detectable by instruments that demonstrably detect the removal of needed information.

6 Requests are billed; restrictions are free

Two things people put in skills were separated on an open-ended dashboard brief, six runs per arm per model: the same 811 words of guidance (**bulk**), and ninety words of restrictive design conventions in the style of real skills — follow established patterns, no decorative elements, restraint with colour — that ask the model to do *less* (**restriction**).

arm	output tokens	latency	file size
minimal	5,114	5	38.7s
restriction (~90 words)	4,951 (−3%)	39.7s (+2%)	14.4k
bulk (811 words)	8,713 (+70%)	79.2s (+104%)	14.0k (−2%)
			25.2k (+76%)

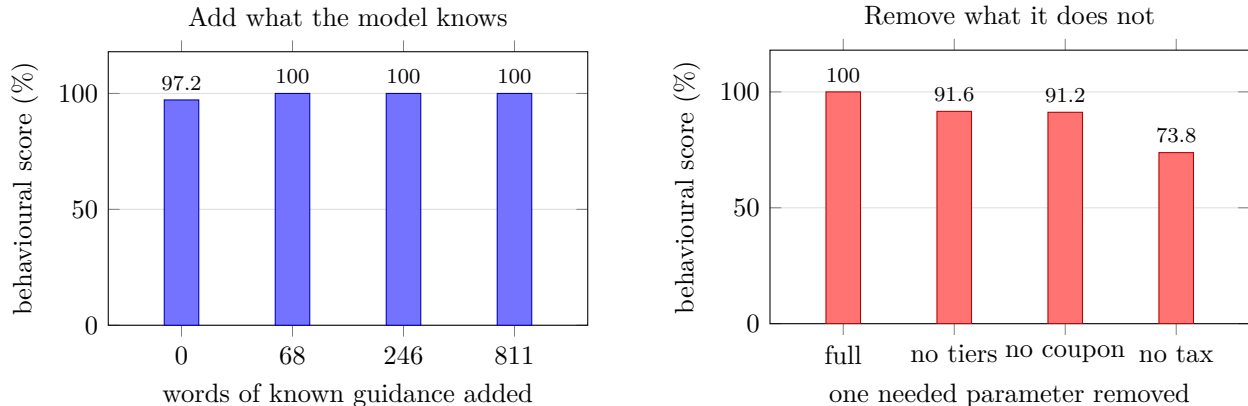


Figure 4: Sensitivity. Left: adding up to 811 words of guidance leaves the ladder tasks at ceiling (97.2% to 100.0%, $n = 12$ per bar). Right: the positive control on the pricing task, where removing a single parameter the model cannot infer drops the score by 26.2 points ($n = 4$ per bar), failing only the checks that depend on it. The pipeline registers a large, targeted loss of information; its power against a small diffuse effect is bounded in §5.

	subtotal	discount	shipping	tax	total
full	100%	100%	100%	100%	100%
no tiers	100%	84%	100%	84%	84%
no coupon	100%	86%	100%	86%	86%
no tax	100%	100%	100%	25%	25%

Figure 5: Where the failures land. Pass rate of the positive control’s five check families under each ablation ($n = 4$ runs per arm, 14 checks per family per run). Removing the tax rate breaks only tax and total; subtotal, discount and shipping stay at 100%. Removing the tiers or the coupon rules breaks the discount-dependent families and nothing else.

separate from baseline. **What the runs looked like.** On the model that started with the most range, six uninstructed runs chose five distinct accent colours and two typefaces; under either bulk or restriction they chose two colours and one face. The other model leaned blue from the start and became *less* alike under both arms — on all three components of the metric under bulk, and on two of three under restriction, the third unchanged. This is a descriptive pattern at $n = 6$ and is reported as a lead, not a result (Figure 7); a length-invariant convergence metric with its own positive-control arm is the design that would settle it.

7 A validation protocol for behavioural graders

Every finding above rests on graders that drive generated pages like a user. Six defects were found in our own graders over the course of the study, and each produced a clean, plausible, wrong number. They motivate three controls that we now apply before any model output is judged, and that we recommend for any behavioural evaluation of generated code.

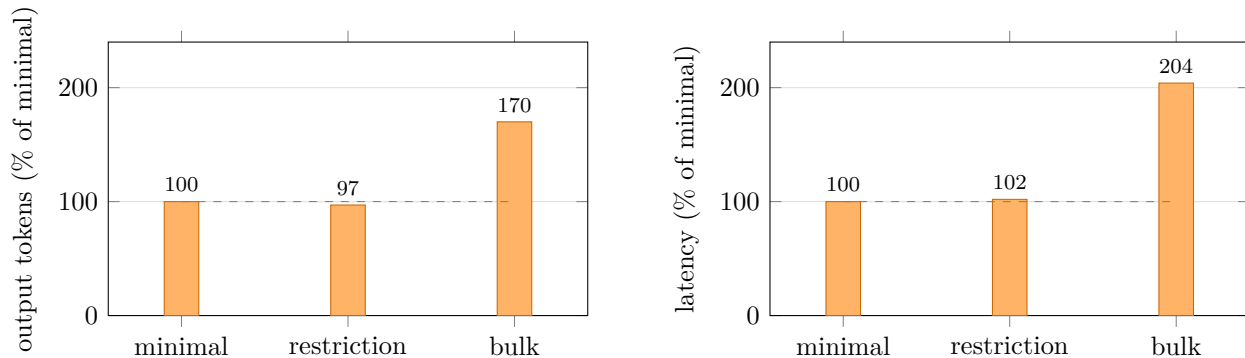


Figure 6: Bulk costs; restriction does not. Open-ended dashboard brief, $n = 12$ per bar over two models. Ninety words of restrictive rules change output tokens by -3% and latency by +2%. Eight hundred and eleven words of generic engineering guidance – none of it wrong, none of it needed – change them by +70% and +104%.

The three controls. A hand-written *correct* implementation of the task, which the grader must score 100%; an *empty* page, which it must score 0%; and a *single-rule mutation* of the correct implementation, which must fail exactly the checks that name that rule and no others. Together they establish that the grader recognises correct work, passes nothing by default, and observes each behaviour it claims to test.

What they would have caught. The table task scored exactly 8 of 10 in all 24 runs — constant across two models and four levels, the signature of a broken check. Two checks were failing deterministically: one clicked the `<th>` while every generated file attached its sort handler to a `<button>` inside it, and events propagate up, not down; the other compared rows visible on page one, where a filter that reduces 24 records to 19 still shows 10. Corrected, all 24 files score 10 of 10. A rule checker double-counted every hex colour because `<style>` sits inside `<body>`, and scored compliant `:root` definitions as violations. A visible-text check read hidden nodes through `textContent`, inverting an early pilot. A missing element read as an empty string, indistinguishable from a valid field, so an empty page scored 6 of 31. And our own reference implementation parsed an ISO date as UTC midnight, shifting it a day earlier under local `getDate()` — the check that caught it was right and the reference was wrong, which is the direction a validated instrument is supposed to resolve. Each of the first four was found only by asking why a number was suspiciously stable; each of the last two was found by the controls at once.

8 Related work

A substantial literature optimises prompts automatically — DSPy (Khattab et al., 2023), GEPA (Agrawal et al., 2025), SPRIG (Zhang et al., 2024), SePO (Tao et al., 2026), SPEAR (Lu et al., 2026) and *p1* (Gao et al., 2026) search for wording that scores better, and MAS-PromptBench (Bai and Shi, 2026) asks when that helps in multi-agent settings. That work asks what the best prompt is; we ask what correct but unrequested content costs and buys, which optimisation presupposes.

The harm hypothesis has empirical precedent on earlier models: irrelevant context measurably distracts them (Shi et al., 2023), longer inputs alone degrade reasoning (Levy et al., 2024), and information in the middle of a long context is used less (Liu et al., 2023). Our ceiling result on current models is positioned against that precedent rather than in ignorance of it: the guidance

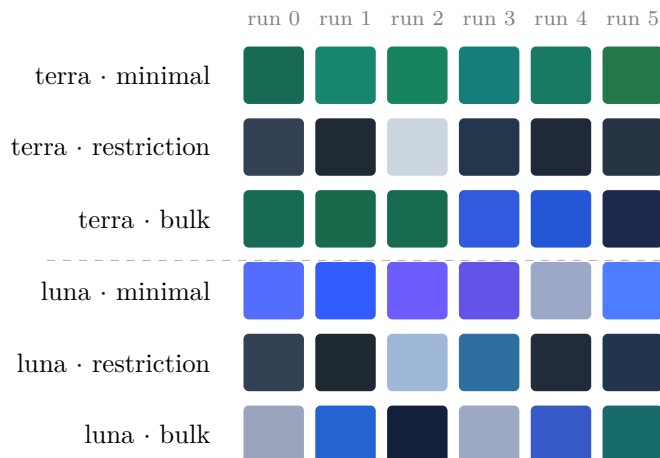


Figure 7: The dominant accent colour each run chose, extracted from its CSS. Six uninstructed runs of terra (top) span 6 distinct accents; under ninety words of restriction or 811 words of bulk they narrow to 6 and 6. Luna (bottom) leans blue from the start (6 / 6 / 6) and has little range to lose. A secondary measure at $n = 6$: a lead, not a result.

here is relevant and correct, the tasks are short, and the project rules were placed first. Prompt compression (Jiang et al., 2023) is the standard remedy for input cost; our finding that the bill lands on the output, and that the output growth is requested features rather than verbosity, is the case compression does not address. Instruction-following benchmarks (Zhou et al., 2023) establish that current models follow checkable constraints reliably, which our compliance result corroborates; sensitivity to spurious prompt features (Sclar et al., 2023) is a related but distinct concern. Finally, the argument that agent evaluations should treat cost as a first-class metric alongside accuracy (Kapoor et al., 2024) is one this paper takes literally. Both vendors publish prescriptive prompting guidance (Anthropic, 2026; OpenAI, 2026), and skills accumulate general material of the kind measured here by imitation of it.

9 Limitations

Two sibling models from one vendor, one task family (self-contained front-end builds), $n = 3$ per cell in the ladder and $n = 1$ in the real-skill pilot, and a correctness instrument at ceiling whose minimum detectable effect is stated in §5. Reported output tokens include reasoning tokens; file size is the visible proxy. Latency in the ladder was not protected against run-order drift, so tokens are the primary cost measure. The zero-guidance arm still carries the agent harness's own base prompt. The ladder reaches 811 words while the motivating skill is 6,282; the pilot at that dose is $n = 1$. Designs were fixed in code before their runs but not externally registered. The convergence question is open: a length-invariant metric with a positive-control arm, more than six runs per group, and a second vendor are the obvious next study.

10 What follows for practice

The defensible recommendation is short, and it is not "shorter is better."

Every generic line is a purchase order. The model will not apply it unprompted, will apply it when asked, and will bill for it: 42% more output tokens and 38% more latency at 811

words on a fixed task, twice the latency on an open-ended one, and up to 38% more on a real 843-line skill.

Keep what you want to buy. Reduced-motion support, ARIA, responsive units and empty-state handling are real features with real users. If a team wants them, the guidance is worth its cost, and the cost is now known. If it does not, the same lines are pure overhead.

Restrictions are free; requests are billed. Rules that ask for restraint cost nothing. Rules that ask for work cost as much as guidance, however they are phrased.

A procedure. List each generic line in the skill. Run the task once at zero guidance and check mechanically which lines are already satisfied — the five-rule compliance test in §5 is that tool. Delete the satisfied ones. For each remaining line, decide whether its output is wanted. Re-measure output tokens. Repeat at the next model release, because the boundary between what a model does unprompted and what it does when asked moves with every one.

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.

Anthropic. Prompting best practices. <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>, 2026. Harvested 2026-08-30; re-diffed 2026-09-04.

Juyang Bai and Laixi Shi. MAS-PromptBench: When does prompt optimization improve multi-agent llm systems? *arXiv preprint arXiv:2606.23664*, 2026.

Zhaolin Gao, Yu, Wang, Bo Liu, Thorsten Joachims, Kianté Brantley, and Wen Sun. p1: Better prompt optimization with fewer prompts. *arXiv preprint arXiv:2604.08801*, 2026.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, et al. LLMlingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736*, 2023.

Sayash Kapoor, Benedikt Stroebl, Zachary S. Siegel, et al. AI agents that matter. *arXiv preprint arXiv:2407.01502*, 2024.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, et al. DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.

Mosh Levy, Alon Jacoby, and Yoav Goldberg. Same task, more tokens: the impact of input length on the reasoning performance of large language models. *arXiv preprint arXiv:2402.14848*, 2024.

Nelson F. Liu, Kevin Lin, John Hewitt, et al. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023.

Mengyin Lu, Cong Feng, Huimin Han, Guangming Lu, Yu Sun, Xiaonan Ding, Shihui Long, Fengyi Li, and Tanvi Motwani. SPEAR: Code-augmented agentic prompt optimization. *arXiv preprint arXiv:2605.26275*, 2026.

OpenAI. Prompt engineering. <https://developers.openai.com/api/docs/guides/prompt-engineering>, 2026. Harvested 2026-08-30; re-diffed 2026-09-04.

- Melanie Sclar, Yejin Choi, Yulia Tsvetkov, et al. Quantifying language models’ sensitivity to spurious features in prompt design. *arXiv preprint arXiv:2310.11324*, 2023.
- Freda Shi, Xinyun Chen, Kanishka Misra, et al. Large language models can be easily distracted by irrelevant context. *arXiv preprint arXiv:2302.00093*, 2023.
- Wangcheng Tao, Han Wu, and Weng-Fai Wong. SePO: Self-evolving prompt agent for system prompt optimization. *arXiv preprint arXiv:2606.04465*, 2026.
- Lechen Zhang, Tolga Ergen, Lajanugen Logeswaran, Moontae Lee, and David Jurgens. SPRIG: Improving large language model performance by system prompt optimization. *arXiv preprint arXiv:2410.14826*, 2024.
- Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, et al. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.